**CHRIST (DEEMED TO BE UNIVERSITY)**

**DELHI NCR CAMPUS • SCHOOL OF SCIENCES**

# ACADEMIC PROJECT REPORT

## EVENT VOLUNTEER HOUR LEDGER

### A Relational Database Management System for Service Hour Accounting, Faculty Verification & Certificate Eligibility

**Course:** Database Management Systems (DBMS) Laboratory & Theory
**Academic Year:** 2025–2026 | **Degree:** Bachelor of Science / Computer Applications
**Student Organization:** AADIKALA (Cultural, Technological and Performing Arts Wing)
**Sub-Clubs:** Dance, Emcee, Technical, Creative, Music, Hospitality
**Live System Deployment:** https://radhika2405-max.github.io/event-volunteer-hour-ledger/

---

**CERTIFICATE OF BONAFIDE WORK**

This is to certify that this project report entitled **'Event Volunteer Hour Ledger'** is a bonafide record of academic work carried out by the student under the School of Sciences, CHRIST (Deemed to be University), Delhi NCR Campus. The implementation adheres to Third Normal Form (3NF), enforces ACID transactions, and provides authoritative student service certificate governance.

_____ _____

**Faculty Mentor / Guide** **Head of Department**

**ABSTRACT**

The **Event Volunteer Hour Ledger** is a relational database management application developed to track student service contributions across university events, calculate verified cumulative hours, and automatically determine eligibility for official university volunteer certificates. Traditional paper timesheets or unnormalized spreadsheets suffer from mutable update anomalies, lost updates during concurrent edits, double-counting in multi-club collaborations, and unverified certificate generation. By structuring volunteer contributions as an immutable transactional ledger in MySQL 8.0 normalized to 3NF/BCNF, the system guarantees ACID transactions, prevents join inflation, enforces engine-level integrity constraints, automates change-data audit logging via triggers, and provides sub-millisecond query execution via balanced B-tree indexing. The application is deployed live with a React + Vite frontend and Python Flask REST API.

## 1. Introduction & Problem Statement

Campus life at CHRIST University Delhi NCR thrives on student volunteer leadership across academic conferences, cultural symposia, and community outreach. The umbrella student organization **AADIKALA** coordinates six specialized sub-clubs: Dance, Emcee, Technical, Creative, Music, and Hospitality. Organizing campus-wide festivals like *SUNERGOS* and *TARANG* requires coordinated student labor spanning setup, stage coordination, and guest hospitality.

Historically, service hours were tracked using manual paper forms and shared cloud spreadsheets. This traditional approach exhibited four fatal database flaws:

1. **Mutable Aggregates & Lost Updates:** Total hours were stored as a single editable number on student records. When multiple coordinators approved hours concurrently, uncoordinated writes caused lost updates without an audit trail.
2. **Multi-Club Join Inflation:** Campus events are frequently co-hosted by multiple sub-clubs (e.g. Technical and Hospitality). In unnormalized spreadsheets or queries, students co-registered with multiple clubs had their hours multiplied by the number of participating clubs.
3. **Premature Certification:** Students with pending, unverified timesheets claimed certificates before faculty review due to the lack of status isolation.
4. **Complete Absence of Auditability:** Modifications, rejections, and approvals lacked Change Data Capture (CDC), preventing administrative traceability.

## 2. Objectives & System Scope

The primary academic and technical objectives of this project are:

- Engineer a 3NF/BCNF relational database schema in MySQL 8.0 eliminating all data anomalies.
- Implement the **Ledger Paradigm**: Hours are stored as discrete, verified entries rather than mutable totals.
- Guarantee mathematical accuracy through SQL views with conditional aggregation, resolving join inflation.
- Enforce business rules via active database triggers (e.g., mandatory rejection reasons with SQL error 45000).
- Enforce ACID compliance in stored procedures using row-level locking (SELECT ... FOR UPDATE).
- Deliver a filterable Certificate Eligibility Report engine and public SHA-256 certificate verification portal.

| In Scope | Out of Scope |
|---|---|
| • User authentication & RBAC (5 roles)<br>• Interval hour logging with break deduction<br>• Multi-club event association<br>• Automated audit trails via triggers<br>• Real-time eligibility evaluation<br>• PDF certificate & CSV report generation<br>• Public cryptographic token verification | • Physical biometric hardware integration<br>• University fee/ERP financial payment gateways<br>• Native iOS/Android compiled binaries (web PWA deployed) |

## 3. System Architecture

The application employs a decoupled Three-Tier Architecture separating user interface, business API logic, and relational persistence:

| Tier | Technology Stack | Responsibilities |
|---|---|---|
| **Presentation Tier** | React 18, Vite, TypeScript, Tailwind CSS, Recharts (Hosted on GitHub Pages) | Interactive Volunteer Portal, Approver Hub, Eligibility Engine, Searchable Reports with CSV export, and Public Token Verifier. |

| | | |
|---|---|---|
| **Application Tier** | Python 3.14, Flask REST API, Flask-JWT-Extended, PyMySQL, ReportLab | Role-Based Access Control (RBAC), token issuance, JSON serialization, PDF rendering, and parameterized query execution. |
| **Database Tier** | MySQL 8.0 InnoDB Engine | 10 Normalized Tables in 3NF, 5 Specialized Views, Triggers, Stored Procedures, Check Constraints, B-Tree Indexes. |

## 4. Relational Data Modeling & Entity-Relationship Design

The database schema is engineered across 10 normalized tables. The Entity-Relationship structure captures real-world university relationships with rigorous cardinalities:

| Entity / Table | Primary Key | Foreign Keys | Cardinality & Purpose |
|---|---|---|---|
| users | user_id | None | 1:1 with students; authentication identity and system roles. |
| students | student_id | user_id (Unique) | 1:1 with users; stores university registration number, degree programme. |
| clubs | club_id | parent_club_id | 1:N Recursive Self-Join; models parent AADIKALA and child sub-clubs. |
| events | event_id | created_by | 1:N with participation; campus events with dates, venues, statuses. |
| event_subclubs | (event_id, club_id) | event_id, club_id | M:N junction connecting events to collaborating sub-clubs. |
| event_volunteers | participation_id | event_id, student_id | M:N associative entity; UNIQUE (event_id, student_id). |
| volunteer_hour_entries | hour_entry_id | participation_id | 1:N with participation; transactional ledger of work intervals. |
| certificate_policies | policy_id | None | 1:N with certificates; sets threshold (e.g. 1,200 mins = 20 hrs). |
| certificates | certificate_id | student_id, policy_id | 1:N with students; immutable award snapshot with SHA-256 token. |
| audit_logs | audit_id | actor_user_id | 1:N with users; captures before/after JSON states via triggers. |

## 5. Database Normalization Analysis (1NF $\to$ 3NF & BCNF)

Normalization was conducted systematically to eliminate redundancy while preserving functional dependencies:

**5.1 First Normal Form (1NF):** Enforced atomic attribute domains. Multi-club collaborations previously stored as sub_clubs = 'Dance, Creative, Technical' violated 1NF. This was decomposed into the associative relation event_subclubs(event_id, club_id) with composite primary key (event_id, club_id).

**5.2 Second Normal Form (2NF):** Removed partial functional dependencies. In composite keys such as event_subclubs, no non-key attribute depends on a subset of the primary key. Club attributes (club_name) depend strictly on club_id, and event attributes (venue, dates) depend strictly on event_id.

**5.3 Third Normal Form (3NF) & BCNF:** Eliminated transitive dependencies ($X \to Y \to Z$). Login credentials (password_hash, role) were isolated into users, while academic data (programme, registration_number) resides in students. Crucially, derived total approved hours was eliminated from students because it represented a transitive dependency on the state of volunteer_hour_entries. Total hours are dynamically evaluated via SQL views.

## 6. Database Integrity Constraints

Integrity rules are enforced directly by the MySQL storage engine, guaranteeing mathematical and temporal realism:

| Constraint Type | Implementation Code | Enforced Business Rule |
|---|---|---|
| **CHECK Constraint** | `CHECK (end_datetime > start_datetime)` | Prohibits events ending before their start timestamp. |
| **CHECK Constraint** | `CHECK (break_minutes >= 0)` | Prohibits negative break durations. |
| **CHECK Constraint** | `CHECK (claimed_minutes > 0)` | Ensures volunteer timesheets record positive work duration. |
| **UNIQUE Constraint** | `UNIQUE (event_id, student_id)` | Prevents duplicate registrations for the same event. |
| **UNIQUE Constraint** | `UNIQUE (verification_token)` | Guarantees uniqueness of public certificate validation hashes. |
| **Referential Action** | `ON DELETE CASCADE` | Cascades teardown from registrations to logged time intervals. |
| **Referential Action** | `ON DELETE RESTRICT` | Protects issued certificates and policies from accidental deletion. |

# 7. Advanced DBMS Implementation & Techniques

## 7.1 Database Views & Resolving Join Inflation

A common pitfall in volunteer management is **Join Inflation**: when an event has 3 co-organizing sub-clubs in event_subclubs, joining events directly with hour entries multiplies every recorded hour by 3. Our solution decouples club association from participation and utilizes 5 dedicated SQL views:

1. vw_student_approved_hours: Computes canonical approved, pending, and rejected hours per student using conditional SUM(CASE WHEN...).
2. vw_certificate_eligibility: Compares student totals against the active threshold policy (1,200 mins = 20 hrs), isolating eligibility from issuance.
3. vw_event_volunteer_summary: Headcount and verified labor per event without duplicate club rows.
4. vw_subclub_contribution_summary: Hierarchy-aware analytics for parent AADIKALA and child sub-clubs.
5. vw_pending_review_queue: Exposes pending submissions to faculty approvers without exposing sensitive password hashes.

## 7.2 Active Database Triggers (Workflow & Audit Logging)

Triggers enforce business invariants and Change Data Capture (CDC):

- **trg_vhe_before_insert:** Validates input durations before row insertion.
- **trg_vhe_before_update:** State machine gatekeeper. If a faculty member attempts to reject hours without providing a written reason, it raises SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Integrity Error: Rejection requires a clear rejection reason.'.
- **trg_vhe_after_update:** Automated audit logger. When hour status or minutes change, it captures old and new values as JSON objects and writes them into audit_logs.

## 7.3 Stored Procedures & ACID Concurrency Control

Certificate issuance is executed via stored procedure sp_issue_certificate wrapped in an explicit ACID transaction:

- **Atomicity:** Validation checks, certificate row creation, and audit logging succeed as one unit or roll back completely.
- **Consistency:** The active policy threshold (1,200 mins) must be satisfied before commit.
- **Isolation:** Uses row-level exclusive locks: SELECT ... FOR UPDATE on the candidate student row and FOR SHARE on the policy row, preventing concurrent duplicate certificate generation.
- **Durability:** Committed transactions are guaranteed by InnoDB write-ahead redo logging.

## 7.4 Indexing Strategy & Query Optimization

To guarantee high throughput across thousands of records, we implemented a targeted B-Tree indexing strategy:

- **Primary Clustered Indexes:** On all auto-increment primary keys for $O(\log N)$ lookups.
- **Secondary B-Tree Indexes:** idx_users_email, idx_students_reg_no, idx_cert_verification_token enable sub-millisecond searches.
- **Composite Indexes:** idx_vhe_part_status (participation_id, submission_status) accelerates view joins. EXPLAIN analysis confirms queries use index seeks (ref/range) rather than full table scans (ALL).

## 8. Analytical Reporting Queries & SQL Demonstrations

The project implements 6 advanced analytical queries in database/reports.sql showcasing advanced SQL features:

**Query 1: Identify Students Eligible for Certificate (>= 20 Hours Threshold)**
*Demonstrates:* INNER JOIN, SUM with CASE, GROUP BY, HAVING with subquery.

```
SELECT s.university_registration_number, u.full_name, ROUND(SUM(vhe.verified_minutes)/60.0, 2) AS approved_hours
FROM students s INNER JOIN users u ON s.user_id = u.user_id
INNER JOIN event_volunteers ev ON s.student_id = ev.student_id
INNER JOIN volunteer_hour_entries vhe ON ev.participation_id = vhe.participation_id
WHERE vhe.submission_status IN ('approved', 'corrected')
GROUP BY s.student_id, s.university_registration_number, u.full_name
HAVING SUM(vhe.verified_minutes) >= (SELECT minimum_approved_minutes FROM certificate_policies WHERE status='active' LIMIT 1);
```

**Query 2: Students Close to Eligibility (Within 5 Hours of Threshold: 15–19.99 Hrs)**
*Demonstrates:* CROSS JOIN with policy configuration, HAVING range comparison.
*Business Purpose:* Enables coordinators to identify and encourage near-threshold volunteers to participate in upcoming events.

**Query 3 (Event-Wise Labor without Duplication):** Aggregates distinct volunteer counts and verified hours per event.
**Query 4 (Sub-Club Contribution Breakdown):** Self-joins on hierarchical clubs table to measure labor across Dance, Emcee, Technical, Creative, Music, Hospitality.
**Query 5 (Pending Hours Impact Simulation):** Projects which students will cross the 20-hour mark if their pending timesheets are approved.
**Query 6 (Certificate Audit Trail):** Comprehensive traceability linking certificate numbers, recipients, issuing faculty, and SHA-256 tokens.

## 9. Full-Stack Implementation & Live Deployment

The system has been completely implemented and hosted live on GitHub Pages:

**LIVE APPLICATION URL:** https://radhika2405-max.github.io/event-volunteer-hour-ledger/
- **Volunteer Portal:** Shows logged intervals, real-time verified progress, and registered events.
- **Approver Hub:** Faculty interface for approving hours, issuing corrections, and rejecting invalid claims.
- **Eligibility Engine:** Interactive threshold bar (>= 20 hours) and multi-filter report with CSV export.
- **Public Verifier:** Unauthenticated SHA-256 token validator verifying credential authenticity.

## 10. Testing, Verification & Test Scenarios

The system was evaluated against automated test suites and realistic volunteer scenarios:

- **Student A (18 Hours Approved, 0 Pending):** Correctly classified as Not Eligible (Needs 2.0 hours).
- **Student B (20 Hours Approved, 0 Pending):** Correctly classified as Eligible - Awaiting Approval.
- **Student C (18 Hours Approved, 5 Hours Pending):** Correctly classified as Pending Hours Review; simulation confirms projected eligibility.
- **Student D (24 Hours Approved, Certificate Issued):** Correctly classified as Certificate Issued with immutable snapshot hours.

## 11. Conclusion & References

The **Event Volunteer Hour Ledger** successfully applies database management system theory to solve a critical campus administrative challenge for CHRIST (Deemed to be University). By strictly enforcing 3NF normalization, ACID transaction boundaries, view-based dynamic aggregation, and role-based access control, the system eliminates data anomalies and ensures reliable academic certificate governance.

**References:**

1. Elmasri, R., & Navathe, S. B. *Fundamentals of Database Systems*, 7th Edition, Pearson.
2. Silberschatz, A., Korth, H. F., & Sudarshan, S. *Database System Concepts*, 7th Edition, McGraw-Hill.
3. MySQL 8.0 Reference Manual — Stored Routines, Views, Triggers, and InnoDB Transaction Model.
4. Flask Documentation & SQLAlchemy 2.0 Unified Documentation.